

EP-1: Gap on Core Concept

Mutated Question (HumanEval 26)

```
def remove_duplicates(numbers: List[int]) -> List[int]:  
    """ From a list of integers, remove all elements that occur  
    more than once. Keep order of elements left the same as in the  
    input.  
    >>> remove_duplicates([1, 2, 3, 2, 4])  
    [1, 3, 4]  
    """
```

Plan

- Remove duplicates while maintaining order
- Avoid external libraries for uniqueness checks
- Ensure efficient performance for large datasets
- Maintain simplicity and clarity in code
- Provide accurate results consistently

We will implement a function that iterates through the list and uses a set to track seen elements, ensuring that duplicates are removed and the original order are maintained. This approach does not need external libraries for uniqueness checks and is efficient for large datasets.

Code

```
def remove_duplicates(numbers: List[int]) -> List[int]:  
    """Removes duplicates from a list of integers while maintaining the  
    order of first occurrence.  
  
    Args:  
        numbers (List[int]): A list of integers which may contain  
        duplicates.  
  
    Returns:  
        List[int]: A list of integers with duplicates removed,  
        maintaining the order of first occurrence.  
    """  
    seen = set() # A set to track numbers that have already been  
    encountered  
    result = [] # A list to store the numbers without duplicates  
  
    for number in numbers:  
        if number not in seen:  
            seen.add(number) # Add the number to the set of seen numbers  
            result.append(number) # Append the number to the result list  
  
    return result
```

EP-2: Gap on Edge Cases

Mutated Question (HumanEval 18)

```
def how_many_times(string: str, substring: str) -> int:
    '''Find how many times a given substring can be found in the
    original string. Consider each appearance of the substring, even
    if it overlaps with another occurrence. Count overlapping cases.
    >>> how_many_times('a', '')
    1
    >>> how_many_times('aaa', 'a')
    3
    >>> how_many_times('aaaa', 'aa')
    3
    ...
```

Plan

```
"subproblems":
    "Identify the length of the original string",
    "Identify the length of the substring",
    "Iterate through the original string to find occurrences of
the substring",
    "Count the number of occurrences of the substring"
    "handle the cases of empty string"
"high-level steps":
    "Get the original string and substring as input",
    "Initialize a counter variable to keep track of the number of
occurrences",
    "Iterate through the original string using a sliding window
approach",
    "Check if the current substring matches the given substring",
    "Increment the counter if a match is found",
    "Return the final count of occurrences"
```

Code

```
def how_many_times(string: str, substring: str) -> int:
    if not string or not substring:
        return 0

    count = 0
    for i in range(len(string) - len(substring) + 1):
        if string[i:i + len(substring)] == substring:
            count += 1

    return count
```

EP-3: Gap on Complex Logic

Mutated Question (HumanEval 87)

```
def get_row(lst, x):
    '''You are given a two dimensional data, as a nested lists, which
    is similar to matrix. However, unlike matrices, each row may
    contain a different number of columns. Given lst, and integer x,
    find integers x in the list, and return a list of tuples, [(x1,
    y1), (x2, y2) ...] such that each tuple is a coordinate - (row,
    columns), starting with 0. Sort coordinates initially by rows in
    ascending order. Also, sort coordinates of the row by columns in
    descending order.
    Examples:
        get_row([
            [1,2,3,4,5,6],
            [1,2,3,4,1,6],
            [1,2,3,4,5,1]
        ], 1) == [(0, 0), (1, 4), (1, 0), (2, 5), (2, 0)]
        get_row([], 1) == []
        get_row([[], [1], [1, 2, 3]], 3) == [(2, 2)]
    '''
```

Plan

```
"subproblems"
    "Identify the rows in the 2D data that contain the integer x",
    "Sort the coordinates initially by rows in ascending order",
    "Sort the coordinates of each row by columns in descending
order"
"major_steps":
    "Iterate through each row in the 2D data",
    "For each row, check if it contains the integer x and store
the coordinates",
    "Sort the coordinates by rows in ascending order",
    "Sort the coordinates of each row by columns in descending
order",
    "Return the sorted list of coordinates"
```

Code

```
def get_row(lst, x):
    coordinates = []

    for i, row in enumerate(lst):
        if x in row:
            for j in range(len(row)):
                if row[j] == x:
                    coordinates.append((i, j))

    coordinates.sort(key=lambda coord: coord[0])
    for i in range(len(coordinates)):
        coordinates[i] = (coordinates[i][0], coordinates[i][1])

    coordinates.sort(key=lambda coord: coord[1], reverse=True)

    return coordinates
```

Mutated Question (CodeContest 1619_A)

A string is considered square if it is some string repeated two times; for example, the strings "aa", "abcabc", "abab" and "baabaa" are square, but the strings "aaa", "abaaab" and "abcdabc" are not square. For a given string s determine if it is square.

Input

The first line of input data contains an integer t ($1 \leq t \leq 100$) —the number of test cases.

This is followed by t lines, each containing a description of one test case. The given strings consist only of lowercase Latin letters and have lengths between 1 and 100 inclusive.

Output

For each test case, output on a separate line:

- * YES if the string in the corresponding test case is square,
- * NO otherwise.

Plan

```
"subproblems":
  "Read the number of test cases from input",
  "For each test case, read the string from input",
  "Check if the string is square by checking if it can be formed
by repeating a substring twice",
  "Output 'YES' if the string is square, 'NO' otherwise"
"high_level_plan":
  "Step 1": "Read the number of test cases from input",
  "Step 2": "For each test case, read the string from input",
  "Step 3": "Check if the string is square by checking if it can
be formed by repeating a substring twice",
  "Step 4": "Output 'YES' if the string is square, 'NO'
otherwise"
```

EP-4: Gap on Relational Phrases

Code

```
def is_square_string(s):
    n = len(s)
    for i in range(1, n // 2 + 1):
        if n % i == 0 and s[:i] * (n // i) == s:
            return "YES"
    return "NO"
```

EP-5: Gap on Condition Judgments

Mutated Question (HumanEval 128)

```
def prod_signs(arr):  
    '''You are given an array arr of integers and you need to return  
    sum of magnitudes of integers multiplied by product of all signs  
    of each number in the array, represented by 1, -1 or 0. Note:  
    return None for empty arr.  
    Example:  
    >>> prod_signs([1, 2, 2, -4]) == -9  
    >>> prod_signs([0, 1]) == 0  
    >>> prod_signs([]) == None  
    ...
```

Plan

1. Check if the input array is empty.
2. Calculate the product of all signs in the array.
3. Calculate the sum of magnitudes of integers in the array.
4. Multiply the product of signs with the sum of magnitudes.

Code

```
def prod_signs(arr):  
    if not arr:  
        return None  
  
    product_of_signs = 1  
    sum_of_magnitudes = 0  
  
    for num in arr:  
        if num > 0:  
            product_of_signs *= 1  
            sum_of_magnitudes += abs(num)  
        elif num < 0:  
            product_of_signs *= -1  
            sum_of_magnitudes += abs(num)  
        # Here should consider num = 0  
  
    return product_of_signs * sum_of_magnitudes
```